

UNYCO

CRM · Plataforma de Gestão e Reservas

Documento Técnico de Entrega

Arquitetura, Stack, Integrações e Infraestrutura

PROJETO

UNYCO CRM

CLIENTE

UNYCO

ENTREGA

29 de abril de 2026

FORNECEDOR

WESCCTECH · unycoclub.com.br

Sumário

01	Sumário Executivo	2
02	Visão Geral do Sistema	3
03	Arquitetura	4
04	Stack Frontend	5
05	Stack Backend e Banco de Dados	6
06	Integrações Externas	7
07	Central de APIs	8
08	Automações WhatsApp	9
09	Infraestrutura de Produção	10
10	Deploy e CI/CD	11
11	Segurança e Operação	12
12	Suporte, Próximos Passos e Contato	13

1. Sumário Executivo

O UNYCO CRM é uma plataforma fullstack de gestão de relacionamento com o cliente desenvolvida sob medida para a UNYCO. O sistema integra um back-office completo de gestão (CRM), uma landing page pública para reservas de hotéis e um conjunto de integrações com sistemas externos críticos do negócio.

Este documento descreve em detalhes a arquitetura, a stack tecnológica, os módulos funcionais, as integrações com sistemas de terceiros, a infraestrutura de produção e o processo de deploy contínuo da aplicação.

Principais entregas

- CRM completo com gestão de cadastros, planos, pagamentos, reservas, sincronização e usuários
- Landing Page integrada com busca de hotéis, fluxo de reserva e pagamento por Cartão e PIX
- Sincronização bidirecional com o sistema TOTVS (ERP) com tratamento de erros traduzidos
- Central de APIs com monitoramento, configuração dinâmica e teste de endpoints externos
- Automações WhatsApp com construtor visual de fluxos e disparo automático em eventos do sistema
- Infraestrutura containerizada (Docker) com pipeline CI/CD via GitHub Actions e GHCR

Ambiente de produção

- **URL:** <https://unycoclub.com.br>
- **Servidor:** Container Docker em /var/www/html/app-unycoprod
- **Banco de dados:** PostgreSQL (host), database "unycoprod"
- **Repositório:** github.com/Devs-Wescctech/app-unycoprod (branch main)
- **Imagem Docker:** ghcr.io/devs-wescctech/app-unycoprod:latest

2. Visão Geral do Sistema

O UNYCO CRM é organizado em três grandes camadas que compartilham o mesmo backend e banco de dados, mas oferecem experiências distintas conforme o público:

Landing Page (público externo)

Site público acessível na raiz do domínio. Permite que associados busquem hotéis, façam reservas, paguem com Cartão ou PIX e gerenciem suas reservas. Desenvolvido em React com base nos templates HTML da identidade visual UNYCO.

CRM (back-office interno)

Painel administrativo acessado em /crm. Permite ao time UNYCO gerenciar cadastros, planos, pagamentos, reservas, sincronização com TOTVS, automações de WhatsApp e configurações do sistema. Possui controle de acesso por perfil (Administrador, Gerente, Operador, Visualizador).

Backend e integrações

API REST em Node.js/Express que serve frontend e LP, persiste dados em PostgreSQL e atua como camada de integração com sistemas externos: TOTVS (ERP), Coobmais (associados/reservas), Vindi (pagamentos), WESCCTECH WhatsApp e ViaCEP (endereços).

Fluxo principal de uma reserva

- Usuário acessa a Landing Page e busca hotéis filtrando por destino e datas
- Backend consulta a API Coobmais, aplica tarifas por categoria (Silver/Gold/Diamante) e devolve os hotéis com preço já ajustado
- Usuário escolhe apartamento, revisa, confirma e parte para o pagamento
- Pagamento via Vindi (Cartão de Crédito ou PIX com QR Code e polling automático)
- Reserva confirmada dispara automaticamente o fluxo de WhatsApp configurado para o evento booking_confirmed
- Sincronização periódica envia os dados de cadastro ao TOTVS com tratamento e tradução de erros

3. Arquitetura

O sistema adota arquitetura cliente-servidor simples, com frontend SPA em React, backend monolítico em Node.js/Express e banco PostgreSQL. Tudo é empacotado em uma única imagem Docker e servido pelo mesmo processo Node.

Camadas

- **Apresentação:** React 18 (Vite) + Tailwind + Radix UI. Componentes modulares inspirados em shadcn/ui. Páginas LP em /lp/*; CRM em /crm/*.
- **Aplicação:** Express servindo API REST em /api/*, fallback de SPA para o React, e arquivos estáticos da LP (PHP convertidos para HTML/JSX).
- **Persistência:** PostgreSQL acessado via "pg" Pool. Tabelas auto-inicializadas a cada start (tabelas de configuração e tarifas).
- **Integrações:** Camada de proxies dentro do mesmo backend. Tokens e URLs configuráveis em runtime via Central de APIs.
- **Cron / Sync:** Serviço Node interno com cronômetro configurável e logs persistidos em arquivo (sync-data.json) e tabelas auxiliares.

Estrutura do projeto

Layout do repositório

src/ – Frontend React (componentes, contextos, hooks, páginas, serviços, utils)
server/ – Backend Express (index.js, db.js, sync-data.json, api-config.json)
public/ – Assets estáticos (imagens, CSS) e Landing Pages estáticas (/lp)
scripts/ – DDL do banco, guia de deploy e scripts utilitários
.github/ – Pipeline CI/CD (deploy.yml)
Dockerfile · docker-compose.yml · .env.example

Rotas principais

Rota	Função	Auth
/	Landing Page (busca de hotéis)	Público / cookie LP
/home	Alias para LP Home	Público / cookie LP
/lp/index.html	Login / cadastro LP	Público
/lp/checkout.html	Checkout LP	Público
/crm/login	Login do CRM	Público
/crm	Dashboard CRM	Sessão CRM
/crm/Cadastros	Gestão de cadastros	Sessão CRM
/crm/Planos	Gestão de planos (quando ativo)	Sessão CRM + flag
/crm/Sync	Sincronização TOTVS	Sessão CRM
/crm/CentralAPIs	Monitoramento de APIs externas	Sessão CRM
/api/*	Endpoints REST (JSON)	Conforme endpoint

4. Stack Frontend

A camada frontend é totalmente React 18, empacotada com Vite. O CRM é um SPA, e as Landing Pages combinam React com páginas HTML estáticas onde fazia sentido manter a estrutura legada.

Bibliotecas principais

Categoria	Biblioteca	Uso
Build	Vite	Bundler e dev server
UI	Tailwind CSS	Sistema de design utilitário
UI	Radix UI / shadcn-style	Componentes acessíveis (Drawer, Dialog, etc.)
Roteamento	React Router DOM	Roteamento client-side
Dados	TanStack React Query	Cache e sincronização de dados remotos
Formulários	React Hook Form + Zod	Validação e gestão de formulários
Visualização	Recharts	Gráficos do Dashboard
Mapas	Mapa SVG do Brasil	Distribuição por estado (Dashboard, Reservas)
Notificações	Sonner	Toasts e feedbacks
Exportação	xlsx, jsPDF, html2canvas	CSV, Excel e PDF de relatórios
LP estáticas	Bootstrap 5.3, FontAwesome, Flatpickr	Páginas HTML legadas convertidas

Páginas do CRM

- Login, Dashboard, Cadastros, Planos, Pagamentos, Reservas & Receita
- Sincronização TOTVS, Pesquisa TOTVS, Usuários, Configurações
- Automações WhatsApp (com construtor visual de fluxos), Central de APIs

Funcionalidades transversais

- Exportação de qualquer listagem em CSV, Excel e PDF
- Paginação, busca textual e filtros avançados em todas as listagens
- Filtro de período personalizado com calendário (Flatpickr)
- Mapa interativo do Brasil com bolhas por estado, tooltip e modal detalhado

5. Stack Backend e Banco de Dados

Backend único em Node.js/Express. Toda a lógica de negócio, integrações e sincronização ficam em `server/index.js` e em módulos auxiliares. O mesmo processo serve a API e os arquivos estáticos do build do frontend.

Stack

Componente	Detalhe
Runtime	Node.js 20 (Alpine no container)
Framework HTTP	Express
Driver DB	pg (Pool)
Banco de dados	PostgreSQL (host, fora do container)
Persistência auxiliar	JSON em disco para logs de sync e api-config
Health check	GET /api/health (200 ok ou 503 se DB offline)
Auth CRM	Login + perfis (Admin, Gerente, Operador, Visualizador) com credenciais em localStorage
Auth LP	Cookie lp_token gerado após login do associado (Coobmais)

Tabelas principais

Tabela	Função
system_config	Configurações dinâmicas do sistema (plans_enabled, whatsapp_config, etc.)
season_config	Datas de alta e baixa temporada para cálculo de tarifas
category_rates	Tarifas por categoria de hotel (Silver, Gold, Diamante)
Demais tabelas	Dados operacionais do CRM (criadas conforme módulos ativos)

O servidor executa `initializeDatabase()` antes de aceitar tráfego, garantindo que as tabelas críticas existam e que valores padrão estejam presentes. A operação é idempotente (ON CONFLICT DO NOTHING).

Endpoint de health

GET /api/health responde { status: "ok", timestamp } em 200, ou HTTP 503 se a conexão com o PostgreSQL falhar. Usado pelo healthcheck do Docker para reiniciar o container automaticamente em caso de falha.

6. Integrações Externas

Todas as integrações passam pelo backend Node como camada de proxy. Tokens e URLs são configuráveis em runtime via Central de APIs (sem necessidade de redeploy). Configurações são persistidas em `server/api-config.json` (TOTVS, Coobmais, Vindi) e na tabela `system_config` (WhatsApp).

TOTVS (ERP)

- Sincronização de cadastros (CPF, nome, e-mail) com retry automático para erros transitórios
- Tratamento e tradução de erros TOTVS (EXISTCLI, CGC, Loja Inválida) para mensagens em português
- Erros permanentes (CPF inválido) não são reentregues; erros genéricos retentados após 1 hora
- Quando planos estão desativados, sincroniza todos os cadastros com CPF (sem filtro de assinatura)

Coobmais (associados e hotéis)

- Login de associados, listagem de hotéis e gestão de reservas
- Auto-login via AccessKey + password com geração e cache automático do JWT no backend
- Refresh automático do token 5 minutos antes do exp
- UI dedicada na Central de APIs para regenerar token sob demanda

Vindi (pagamentos)

- Ambiente: produção (`app.vindi.com.br`)
- Cartão de Crédito: criação de customer, `payment_profile` e bill
- PIX: geração de QR Code (imagem + Pix Copia e Cola) e polling a cada 5s para confirmação automática
- Produto fixo configurável via `VINDI_PRODUCT_ID` (default 1980987 — "PROJETO UNYCO")

WESCCTECH WhatsApp

- Disparo automático em eventos: `booking_confirmed`, `booking_cancelled`, `registration_completed`
- Construtor visual de fluxos (Trigger, Delay, Message)
- Teste manual com progresso em tempo real e logs persistidos
- Token e URL configuráveis na própria Central de APIs

ViaCEP

- Autopreenchimento de endereços a partir do CEP nos formulários do CRM e da LP

7. Central de APIs

A Central de APIs é o painel de monitoramento e controle de todas as integrações externas. Permite ao time UNYCO operar o sistema sem depender da equipe técnica para alterações de configuração.

Recursos

- Cards de status por API (online, offline, latência) com auto-refresh
- Listagem de endpoints monitorados com teste individual
- Edição de tokens e URLs aplicada em tempo real (sem restart)
- Validação obrigatória de HTTPS, allowlist de domínios, mascaramento de tokens nas respostas
- Coobmais: gestão de AccessKey + password + URL de autenticação, com box read-only do JWT, expiração e botão "Regenerar"

8. Automações WhatsApp

O módulo de automações permite criar fluxos de mensagens WhatsApp que disparam automaticamente em eventos do sistema, sem necessidade de programação.

Construtor visual de fluxos (VisualFlowBuilder)

- Estilo chatbot, com nodes visuais conectados: Trigger !' Delay !' Message
- Cada fluxo é vinculado a um evento de gatilho (booking, cadastro, etc.)
- Mensagens podem usar variáveis dinâmicas (nome, hotel, datas, valor, etc.)

Eventos disparados automaticamente

- **booking_confirmed:** ao salvar uma reserva confirmada
- **booking_cancelled:** ao cancelar uma reserva
- **registration_completed:** ao finalizar cadastro de associado pela LP

Operacional

- Teste de envio com progresso em tempo real e visualização de logs
- Configuração da API WESCCTECH (token e URL) feita pela própria interface
- Persistência da configuração em system_config (key: whatsapp_config)

9. Infraestrutura de Produção

O ambiente de produção roda em um único servidor da WESCCTECH com containerização Docker. O banco PostgreSQL fica no host (fora do container) para garantir isolamento e facilitar backups.

Topologia

Componente	Detalhe
Domínio	unycoclub.com.br
Servidor	Diretório /var/www/html/app-unycoprod
Container	ghcr.io/devs-wescctech/app-unycoprod:latest
Porta exposta	5100 (host) !' 5000 (container)
Banco de dados	PostgreSQL no host, acessado via 172.17.0.1:5432
Database	unycoprod
Healthcheck	GET /api/health (configurado no docker-compose)
Logs	docker logs + arquivos de sync em volume persistente

Empacotamento

- Dockerfile multi-stage: build do frontend (Vite) + cópia para o estágio de runtime Node 20 Alpine
- docker-compose.yml fixa porta 5100 externa, healthcheck e variáveis de ambiente via env_file
- Arquivo .env (chmod 600, fora do repositório) carregado em runtime pelo compose

10. Deploy e CI/CD

A entrega contínua é totalmente automatizada via GitHub Actions. Toda merge na branch main publica uma nova imagem Docker no GitHub Container Registry, pronta para deploy no servidor.

Pipeline (.github/workflows/deploy.yml)

- Trigger: push na branch main
- Build da imagem Docker multi-stage
- Push para ghcr.io/devs-wescctech/app-unycoprod:latest

Atualização do servidor

Comando de update no host

```
cd /var/www/html/app-unycoprod
docker compose pull
docker compose up -d
```

Variáveis de ambiente (produção)

Variável	Função
NODE_ENV	production (definido no compose)
PORT	5000 (definido no compose)
DATABASE_URL	Conexão com o PostgreSQL
DB_SSL	Flag de SSL para o banco
TOTVS_API_TOKEN	Autenticação da API TOTVS
VINDI_API_KEY	Chave de API Vindi
WHATSAPP_API_URL / WHATSAPP_API_TOKEN	API WESCCTECH WhatsApp
COOBMAIS_BASE_URL / AUTH_URL / ACCESS_KEY / PASSWORD	API Coobmais

11. Segurança e Operação

O sistema adota práticas de segurança alinhadas com a sensibilidade dos dados manipulados (CPF, dados de pagamento, dados de associados).

Práticas adotadas

- Tokens de integrações nunca expostos no frontend; sempre proxiadados pelo backend
- Mascaramento automático de credenciais nos retornos da Central de APIs
- Controle de acesso por perfil no CRM (Administrador, Gerente, Operador, Visualizador)
- Rotas sensíveis protegidas (ex.: rota Planos com ProtectedRoute fail-closed)
- Pagamentos processados pela Vindi; cartões nunca trafegam pelo banco da UNYCO
- Validação obrigatória de HTTPS e allowlist de domínios na Central de APIs
- Arquivo .env do servidor com chmod 600 e fora do repositório Git

Tratamento de erros

Sistema centralizado de tradução automática de erros técnicos para mensagens amigáveis em português.

Erros TOTVS são processados primeiro (EXISTCLI/Loja Inválido! "já existe" incompletos", CGC! "CPF inválido"). Códigos HTTP 401/403/502/503/504 utilizados em falsos positivos em mensagens.

12. Suporte, Próximos Passos e Contato

Suporte e operação

- Monitoramento contínuo via `/api/health` e `healthcheck` do Docker
- Alterações de configuração de integrações realizadas pela própria equipe UNYCO via Central de APIs
- Atualizações da aplicação publicadas automaticamente via pipeline GitHub Actions
- Backup do banco PostgreSQL gerenciado no host (fora do container)

Documentação técnica complementar

- `README.md` no repositório com visão geral, comandos e estrutura
- `scripts/db-schema.sql` com o DDL completo do banco
- `scripts/deploy-guide.md` com guia passo a passo de deploy
- `.env.example` com a lista completa de variáveis de ambiente

Contato técnico

WESCCTECH · suporte e desenvolvimento
Repositório: github.com/Devs-Wescctech/app-unycoprod
Produção: <https://unycoclub.com.br>